

claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-a14b8c2401ed820e7.jsonl`  
- SHA-256: `1ee14eabc8d7a908a51477152184685f721e69b6eaf93eccdbdf180c0ee141b6`  
- Source modified: `2026-06-10T06:12:40+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `wf_2a64bc88-2a8`  
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`
```

Transcript

1. user (2026-06-10T06:09:44.588Z)

CONTEXT ? two sibling repos on a Windows machine:

- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).

- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).

Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources. ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.

QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.

ROLE: adversarial verifier. An auditor ("reliability") claimed the finding below. Try to REFUTE it against the actual files. Default to isReal=false if the evidence does not hold up, the behavior is documented/deliberate (e.g. listed in docs/CODE_MAP.md gotchas or DEFERRED_HARDENING_PLAN.md as a known deferral ? unless the auditor adds genuinely NEW consequence), or the severity is materially overstated.

FINDING: {"title":"Segmenter Result-contract is honored by only 1 of 5 segmenters ? every AI failure mode collapses to a silent empty list","severity":"high","area":"silent-failure / pipeline","file":"backend/pipeline/segmenters/base.py:24 (declares Result); gemini.py:36/52/58/263, wasb_hybrid.py:63-166, tracknet_hybrid.py:67-186, yolo_hybrid.py:272-301 (all return []); motion.py:33/226 (only honoror); consumed at backend/main.py:139-150","evidence":"base.process_video is typed `Result[List[...]]` and the docstring tells callers to use `isinstance(result, Failure)` to tell 'no rallies' from 'something broke'. Only MotionPlaySegmenter returns Success/Failure. The four real AI segmenters return a bare `[]` for EVERY distinct failure: missing API key (gemini.py:52, wasb_hybrid.py:73), sport/provider KeyError (gemini.py:36/58), WSL healthcheck fail (wasb_hybrid.py:109, tracknet_hybrid.py:120), detector produced no CSV (wasb_hybrid.py:118), zero candidates (wasb_hybrid.py:152), and genuine no-rallies. In main.py:139 `isinstance(result, Failure)` is therefore dead code for these segmenters; `result.value if hasattr(result,'value')` falls through to the list, and the response says `status:'success', 'Successfully indexed 0 play segments'`.","why_it_matters":"A paying customer whose run failed on a missing key, an exhausted Gemini quota, or a dead WSL GPU env receives an HTTP 200 'success, 0 highlights' that is indistinguishable from a video that genuinely had no rallies. The whole point of the Result type (results.py) is defeated for the production paths.","why":"placeholder"}

Steps: open the cited file(s) and any counter-evidence files; verify quotes/line claims; judge severity honestly. Cheap greps allowed; do NOT run the pipeline. Return isReal, confidence, and

a 1-3 sentence verdict (with corrected detail if partially right).

2. user (2026-06-10T06:09:52.527Z)

```
1      from abc import ABC, abstractmethod
2      from typing import List, Dict, Any, Type, Optional
3      from backend.infrastructure.database import Database
4      from backend.results import Result # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6      class BasePlaySegmenter(ABC):
7          def __init__(self, db: Database, config: Dict[str, Any]):
8              self.db = db
9              self.config = config
10
11          @property
12          @abstractmethod
13          def name(self) -> str:
14              """Unique identifier for this segmenter."""
15              pass
16
17          @property
18          @abstractmethod
19          def description(self) -> str:
20              """Human-readable explanation of this segmenter."""
21              pass
22
23          @abstractmethod
24          def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25              """
26              Processes a video to detect play segments, populates SQLite database,
27              and returns a Result.
28
29              Use isinstance(result, Failure) (or result.is_success()) to distinguish
30              "no segments found" from "something went wrong" (missing key, healthcheck
31              failure, etc.). This is the start of the standardized error/result contract.
32              """
33              pass
34
35      class SegmenterRegistry:
36          def __init__(self):
37              self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39          def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40              """Registers a new segmenter class under its name property."""
41              try:
42                  temp_inst = segmenter_cls(None, {})
43                  name = temp_inst.name
44              except Exception as e:
45                  raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}")
46              self._segmenters[name] = segmenter_cls
47
48          def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49              return self._segmenters.get(name)
50
51          def list_available(self) -> Dict[str, str]:
52              """Returns a dict of registered segmenter names and their descriptions."""
53              res = {}
54              for name, cls in self._segmenters.items():
55                  try:
56                      temp_inst = cls(None, {})
57                      desc = temp_inst.description
58                  except Exception:
```

```

59         desc = "No description available."
60         res[name] = desc
61         return res
62
63     segmenter_registry = SegmenterRegistry()
64

```

3. user (2026-06-10T06:09:53.413Z)

```

100     logger.info(f"Running validations for {req.video_path}...")
101     val_results, val_context = registry.run_all_with_context(req.video_path,
global_config)
102     failures = [r for r in val_results if not r.passed]
103
104     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
105     default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
106     seg_name = req.segmenter or default_seg
107     segmenter_actual = None
108     segmentation_ran = False
109     response: Optional[Dict[str, Any]] = None
110     try:
111         if failures:
112             db_instance.add_video(video_id, req.video_path, "failed", [r.dict() for r in
val_results])
113             response = {
114                 "video_id": video_id,
115                 "status": "failed",
116                 "message": "Video failed validation checks.",
117                 "results": [r.dict() for r in val_results],
118             }
119             return response
120
121         # 2. Run segmenter & indexer
122         print(f"[API] Video passed checks. Segmenting and indexing...")
123         db_instance.add_video(video_id, req.video_path, "processed", [r.dict() for r in
val_results])
124
125         segmenter_cls = segmenter_registry.get(seg_name)
126         if not segmenter_cls:
127             available = list(segmenter_registry.list_available().keys())
128             raise HTTPException(
129                 status_code=400,
130                 detail=f"Segmenter '{seg_name}' not found. Available segmenters:
{available}"
131             )
132
133         print(f"[API] Using segmenter: {seg_name}")
134         segmenter_actual = seg_name
135         segmenter = segmenter_cls(db_instance, global_config)
136         result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
137         segmentation_ran = True
138
139         if isinstance(result, Failure):
140             print(f"[API] Segmenter failed: {result.message}")
141             response = {
142                 "video_id": video_id,
143                 "status": "failed",
144                 "message": f"Segmenter '{seg_name}' failed: {result.message}",
145                 "results": [r.dict() for r in val_results],
146                 "play_segments": [],
147             }
148             return response
149

```

```

150         segments = result.value if hasattr(result, "value") else result
151
152         response = {
153             "video_id": video_id,
154             "status": "success",
155             "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
156             "results": [r.dict() for r in val_results],
157             "play_segments": segments,
158         }
159         return response
160     finally:
161         # Always emit the per-video observability report (next to the video, or to
162         # report.output_dir). Never raises. Surface the paths in the response.
163         paths = emit_indexer_report(
164             db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
165             val_results=val_results, val_context=val_context,
166             segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
167             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
168         )
169         if paths and isinstance(response, dict):
170             response["reports"] = paths
171
172     @app.post("/api/query")
173     def query_play_segments(req: QueryRequest):
174         filters = {
175             "server": req.server,
176             "receiver": req.receiver,
177             "duration_min": req.duration_min,
178             "event_type": req.event_type
179         }

```

4. user (2026-06-10T06:10:00.275Z)

```

1     import os
2     import time
3     import json
4     import logging
5     import subprocess
6     import concurrent.futures
7     from typing import List, Dict, Any, Tuple
8     from backend.infrastructure.database import Database
9     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
10    from backend.utils.video import get_video_duration, extract_video_chunk
11    from backend.sports import sport_registry
12    from backend.providers import provider_registry
13
14    logger = logging.getLogger(__name__)
15
16    class GeminiPlaySegmenter(BasePlaySegmenter):
17        @property
18        def name(self) -> str:
19            return "gemini"
20
21        @property
22        def description(self) -> str:
23            return "High-precision AI-powered semantic play segment detection using Google
Gemini Multimodal API."
24
25        def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
26            """
27            Processes a video to detect play segments using Google Gemini Multimodal Video
API,

```

```

28         populates SQLite, and indexes detailed events.
29         """
30         # Get sport profile config
31         sport_name = self.config.get("sport", "badminton")
32         try:
33             sport_profile = sport_registry.get(sport_name)
34         except KeyError as e:
35             logger.error(str(e))
36             return []
37
38         # Get AI provider and model config
39         idx_cfg = self.config.get("indexing", {})
40         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
41         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
42
43         # Get appropriate API key based on the provider
44         api_key_env_var = f"{provider_name.upper()}_API_KEY"
45         api_key = os.environ.get(api_key_env_var)
46         if not api_key:
47             logger.error(
48                 f"{api_key_env_var} environment variable is not set! "
49                 f"To run the {provider_name} segmenter, set your API key. "
50                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
51             )
52             return []
53
54         try:
55             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
56         except KeyError as e:
57             logger.error(str(e))
58             return []
59
60         # A: Check for debug_duration to trim the video first
61         debug_duration = self.config.get("indexing", {}).get("debug_duration")
62         video_to_upload = video_path
63         is_trimmed = False
64         temp_trimmed_path = None
65         is_compressed = False
66         temp_compressed_path = None
67
68         if debug_duration:
69             logger.info(f"DEBUG FEATURE: Trimming video to first {debug_duration}
seconds for fast processing...")
70             temp_trimmed_path = os.path.join("output", f"temp_trimmed_{video_id}.mp4")
71             os.makedirs("output", exist_ok=True)
72             if os.path.exists(temp_trimmed_path):
73                 try:
74                     os.remove(temp_trimmed_path)
75                 except Exception:
76                     pass
77
78             cmd = [
79                 "ffmpeg", "-y",
80                 "-ss", "0",
81                 "-t", str(debug_duration),
82                 "-i", video_path,
83                 "-c", "copy",
84                 temp_trimmed_path
85             ]
86             logger.debug(f"Running command: {' '.join(cmd)}")
87             try:
88                 subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,

```

```

stderr=subprocess.DEVNULL)
89         logger.info(f"Trimmed video successfully created at:
{temp_trimmed_path}")
90         video_to_upload = temp_trimmed_path

```

5. user (2026-06-10T06:10:00.797Z)

```

55
56     def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None,
57                         max_frames: int = None) -> List[Dict[str, Any]]:
58         sport_name = self.config.get("sport", "badminton")
59         try:
60             sport_profile = sport_registry.get(sport_name)
61         except KeyError as e:
62             logger.error(str(e))
63             return []
64
65         idx_cfg = self.config.get("indexing", {})
66         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
67         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
68
69         api_key_env_var = f"{provider_name.upper()}_API_KEY"
70         api_key = os.environ.get(api_key_env_var)
71         if not api_key:
72             logger.error(f"{api_key_env_var} not set; cannot run the {provider_name}
handoff.")
73             return []
74         try:
75             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
76         except KeyError as e:
77             logger.error(str(e))
78             return []
79
80         video_duration = get_video_duration(video_path)
81         if video_duration <= 0:
82             logger.error("Invalid video duration.")
83             return []
84         fps, frame_width = self._probe_fps_width(video_path)
85
86         wasb_cfg = WasbConfig.from_indexing_cfg(idx_cfg)
87         runner: DetectorRunner = WasbRunner(wasb_cfg)
88
89         w = idx_cfg.get("wasb", {})
90         velocity_thresh = w.get("velocity_threshold", 0.02)
91         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
92         min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
93         handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
94         ai_max_workers = idx_cfg.get("ai_max_workers", 5)
95         log_candidates = idx_cfg.get("log_yolo_candidates", True)
96         store_candidates = idx_cfg.get("store_yolo_candidates", True)
97
98         detection_params = {
99             "detector": "wasb_hybrid",
100             "velocity_thresh": velocity_thresh,
101             "merge_gap": merge_gap,
102             "min_action_window_duration": min_window_duration,
103             "weights_path": wasb_cfg.weights_path,
104         }
105
106         ok, msg = runner.healthcheck()
107         if not ok:

```

```

108         logger.error("WASB WSL environment not ready: %s", msg)
109         return []
110     logger.info("WASB WSL env OK: %s", msg)
111
112     # --- Phase 2: shuttle trajectory -> candidate rally windows ---
113     t_start = time.time()
114     out_dir = os.path.join("output", "wasb")
115     csv_path = runner.run_predict(video_path, out_dir)
116     if not csv_path:
117         logger.error("WASB produced no trajectory; aborting.")
118         return []
119
120     points = runner.parse_trajectory_csv(csv_path)
121     logger.info("Parsed %d trajectory points (%d visible).",
122               len(points), sum(1 for p in points if p.visible))
123
124     all_candidate_windows = runner.trajectory_to_action_windows(
125         points, fps=fps, frame_width=frame_width, chunk_start=0.0,
126         velocity_thresh=velocity_thresh, merge_gap=merge_gap,
127         min_window_duration=min_window_duration, chunk_index=0,
128     )
129     logger.info("Phase 2 (WASB) completed in %.1fs. Candidate windows: %d",
130               time.time() - t_start, len(all_candidate_windows))
131
132     if log_candidates:
133         for cw in all_candidate_windows:
134             logger.info(f"[WASB rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
135                       f"({cw['end'] - cw['start']:.1f}s) | "
136                       f"peak_v={cw['peak_velocity']:.3f} "
137                       f"mean_v={cw['mean_velocity']:.3f}")
138
139     candidate_ids = [None] * len(all_candidate_windows)
140     if store_candidates:
141         for i, cw in enumerate(all_candidate_windows):
142             stats = {
143                 "peak_velocity": cw["peak_velocity"], "mean_velocity":
144                 cw["mean_velocity"],
145                 "active_frame_count": cw["active_frame_count"], "track_id_count":
146                 cw["track_id_count"],
147             }
148             candidate_ids[i] = self.db.add_candidate_segment(
149                 video_id, detector="wasb_hybrid", start_time=cw["start"],
150                 end_time=cw["end"],
151                 chunk_index=cw["chunk_index"], confidence=min(1.0,
152                 cw["peak_velocity"]),
153                 stats=stats, detection_params=detection_params,
154             )
155     if not all_candidate_windows:
156         return []
157
158     # --- Phase 3: AI provider handoff (shared pattern) ---
159     handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
160     os.makedirs(handoff_dir, exist_ok=True)
161     logger.info("Phase 3: %s validation on %d candidate windows...",
162               provider_name, len(all_candidate_windows))
163
164     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
165         w_start = max(0, window["start"] - handoff_padding)
166         w_end = min(video_duration, window["end"] + handoff_padding)
167         w_dur = w_end - w_start
168         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
169         if not extract_video_chunk(video_path, w_start, w_dur, w_path,
170                                   reencode=True):

```

```

166         return []
167     prompt = sport_profile.get_prompt(chunk_duration=w_dur)
168     segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
169                                         label=f"Handoff {wi+1}")
170     valid = []
171     for seg in segments:
172         try:
173             seg["start_time"] = float(seg["start_time"]) + w_start
174             seg["end_time"] = float(seg["end_time"]) + w_start

```

6. user (2026-06-10T06:10:11.462Z)

```

9:from backend.results import Result, Success, Failure # standardized contract
33:         return Failure(message=f"Could not open video: {video_path}",
is_recoverable=False)
226:         return Success(value=segments_data)

```

7. user (2026-06-10T06:10:12.381Z)

```

67:         return []
81:         return []
86:         return []
91:         return []
120:        return []
131:        return []
172:        return []
186:        return []

```

8. user (2026-06-10T06:10:13.831Z)

```

127:        return []
163:        return []
230:        return []
272:        return []
288:        return []
294:        return []
301:        return []
494:        return []

```

9. user (2026-06-10T06:10:14.794Z)

```

36:         return []
52:         return []
58:         return []
263:        return []

```

10. user (2026-06-10T06:10:22.641Z)

```

6-
7-Why these three: they were cataloged as out-of-scope during the consistency
8-pass because each is larger/design-heavy or needs the owner's machine. They are the
9:remaining high-value hygiene items not already done (config/structure/Result/logging/
10-reports) or in flight (detector abstraction, PR #49).
11-
12-Recommended sequence: #7 Packaging (lowest risk, unblocks clean installs/CI) ?

```

11. user (2026-06-10T06:10:24.011Z)

```

124-[Omitted long context line]
125-
126-### 2.3 Eval & calibration (all pure core; I/O in CLI drivers)
127:- `@backend/eval/metrics.py` ? scoring primitive (THE spine). `::interval_iou`,
`::match_intervals` (greedy, deterministic), `::score`->`::Score`(`.as_dict()` = wire shape),
`::pr_curve`, `::Match`, `::MatchResult`. ? greedy not Hungarian; empty matches -> 0.0 (not NaN)

```



```

for mean_iou/boundary errors.
128-- `@backend\eval\annotations.py` ? generic GT loader. `::load_annotations` (`start,end` CSV;
RAISES on bad rows/`start>=end`), `::parse_timestamp` (decimal or MM:SS/HH:MM:SS),
`::write_scaffold`.
129-- `@backend\eval\calibration.py` ? overfit guard. `::cross_validate` (within-video k-fold; ?
defaults `metric='recall'` deliberately), `::leave_one_video_out` (honest cross-video; needs ?2
labelled videos), `::improvement_report` (OPTIMISTIC ? selected on full GT), `::select_best`,
`::kfold_indices`, `::pct_improvement` (? from-zero -> `inf`), `::evaluate`. $ `PredictFn =
Callable[[Config], List[Interval]]` = the plug-in seam. ? `generalization_gap >~0.1` = overfit
alarm.
130-- `@backend\eval\calibrate_wasb.py` ? WASB tuning CLI. `::BASELINE=(0.02,2.0,2.0)`,
`::default_grid` (45 cfgs), `::make_predict_fn` (parses trajectory once, closes over
`trajectory_to_action_windows`), `::load_golden_gts` ($`start_time,end_time` cols; ? SILENTLY
skips bad rows ? opposite of annotations.load_annotations). ? `{load_golden_gts,
make_predict_fn, BASELINE}` imported by 4 downstream drivers (calibrate_local,
export_wasb_segments, gemini_refine, audio/e1_probe).
--
139-[Omitted long context line]
140-[Omitted long context line]
141-[Omitted long context line]
142:[Omitted long matching line]
143-
144-### 2.3a Audio (E1 probe; NOT adopted for fusion ? ADR-007)
145-[Omitted long context line]
--
152-- `@backend\utils\geometry.py` ? `::get_velocity_normalised` (fraction of frame-width/s; ?
raw-px fallback if width<=0; used by yolo_hybrid), `::calculate_iou` (? +1 px convention
inflates IOU), `::get_center/get_distance/get_velocity`.
153-
154-### 2.5 Validators ?
155:[Omitted long matching line]
156-- `@backend\pipeline\validators\__init__.py` ? registers in EXEC order: FormatValidator,
ResolutionFPSValidator, PTSContinuityValidator, SceneCutDensityValidator, BlurValidator,
RelevanceValidator (? producers before consumers ? format_check first; ? import mutates global
registry as side effect).
157-- `@backend\pipeline\validators\format.py::FormatValidator` (`format_check`) ? **PRODUCER**
of `context['ffprobe_meta']`; ? MUST run before resolution_fps; container check is substring
`'mp4'` (so MOV-family passes); external `ffprobe`.
158-- `@backend\pipeline\validators\resolution_fps.py::ResolutionFPSValidator`
(`resolution_fps_check`) ? CONSUMES `ffprobe_meta` (? hard-fails "Run format_check first" if
absent). `min_fps` 29.9; min res 1280x720.
--
205-$ **manifest.json** (cache): `{version, frames_dir(abspath), weights, sport, frames_in,
frames_total, windows_total, windows_done}`. Reuse only if ALL match + `version==CACHE_VERSION`.
206-$ **collector manifest.json** (eval): `{eval_sets:[{video_id(label), local_path, csv},
...]} `; ? DB key = file MD5, manifest `video_id` is a label.
207-
208:$ **eval primitives**: `Interval=Tuple[float,float]` (start<end).
`Match{pred_index,gt_index,iou}`. `MatchResult{matches, unmatched_pred_indices(=FP),
unmatched_gt_indices(=FN)}`. `Score{iou_threshold,num_pred,num_gt,true_positives,false_positives
,false_negatives,precision,recall,f1,mean_iou,mean_start_error,mean_end_error}.as_dict()`.
209-
210-$ **WSL detector candidate** (in-WSL, model<->tracker): `{ 'xy':np.array([x,y]),
'score':float, 'scale':int }`; plain JSON form `[x,y,score,scale]`. ? `_dets_to_tracker` MUST
rebuild `xy` as np.array (tracker does `np.linalg.norm`).
211-

```

12. user (2026-06-10T06:10:32.188Z)

```

3:**Purpose:** cold-start navigation map for the badminton-highlight-indexer ? a self-hosted
video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a
segmenter-agnostic eval/calibration stack, typed config, and per-video observability reporting.
29: -> seg_name = req.segmenter or global_config.indexing.default_segmenter (fallback 'motion')
30: -> seg = segmenter_registry.get(seg_name)(db, global_config) ?
@backend/pipeline/segmenters/base.py::segmenter_registry (400 + available list if missing)

```

```

38:Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same shape):
50:WasbHybridSegmenter.process_video @backend/pipeline/segmenters/wasb_hybrid.py
70:Learned segmenter (offline): training.build_training_set(X,y) ->
classifier.LogisticRegression.fit/save
86: - ? `--segmenter` argparse `choices` frozen at build time from
`segmenter_registry.list_available()` ? a lazily-registered segmenter wouldn't appear. CLI
default_segmenter fallback `motion`.
87:- `@run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths, interactive `input()`
(blocks automation), skips validation. Config via `backend.config.load_config` (typed);
segmenter, padding, and db path all read from the model (no literal fallbacks).
89:- `@run_phase3_eval.py::main` ? walk manifest, segment-if-needed, score, aggregate.
`load_dotenv()` at import. ? DB key = file MD5, NOT manifest's `video_id` (kept as label);
config via `backend.config.load_config`; `segmenter_cls(db, cfg)` receives the typed `AppConfig`
(same object the live pipeline feeds segmenters).
96:[Omitted long matching line]
102:- `@config.json` ? on-disk config; mirrors `AppConfig`. ?
**`indexing.default_segmenter='gemini'`** diverges from model default `yolo_hybrid` ? file
wins at runtime.
105:### 2.1 Segmenters (under pipeline/segmenters) ?
106:- `@backend/pipeline/segmenters/base.py`
107: - `::BasePlaySegmenter` ABC ? `__init__(db, config)`; abstract
`name`, `description`, `process_video`. $ `process_video(video_id, video_path, player_pool=None,
max_frames=None) -> List[dict{id, start_time, end_time, duration, metadata}]`.
108:[Omitted long matching line]
109:- `@backend/pipeline/segmenters/__init__.py` ? imports all 5
(`motion,gemini,yolo_hybrid,tracknet_hybrid,wasb_hybrid`) = the de-facto registration manifest;
re-exports them + `segmenter_registry` via `__all__`.
110:[Omitted long matching line]
111:- `@backend/pipeline/segmenters/tracknet_hybrid.py::TrackNetHybridSegmenter` (name
`tracknet_hybrid`) ? copy-paste twin of wasb_hybrid; imports `TrackNetRunner`/`TrackNetConfig`
from `pipeline/detectors/tracknet_runner`; `indexing.tracknet.velocity_threshold(0.02)`;
`detection_params` adds `queue_length`. ? any P3/P4 fix must be applied to BOTH (and arguably
yolo_hybrid).
112:[Omitted long matching line]
113:- `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter` (name `gemini`) ? pure-AI,
no P2. ? chunks only if `duration>chunk_duration`; `max_workers=5` hardcoded;
`max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates; `parse_time` accepts
colon timestamps (warns); does NOT persist candidate segments (only hybrids feed the fine-tuning
table).
114:- `@backend/pipeline/segmenters/motion.py::MotionPlaySegmenter` (name `motion`) ? pixel
absdiff baseline. ? **metadata + serve/smash/foul/winner events are RANDOM**
(random.sample/randint/uniform/choice); only segmenter using `db.add_event` and passing
server/receiver; uses `print()`; honors `max_frames`.
127:- `@backend/eval/metrics.py` ? scoring primitive (THE spine). `::interval_iou`,
`::match_intervals` (greedy, deterministic), `::score`->`::Score`(`.as_dict()` = wire shape),
`::pr_curve`, `::Match`, `::MatchResult`. ? greedy not Hungarian; empty matches -> 0.0 (not NaN)
for mean_iou/boundary errors.
133:- `@backend/eval/batch.py` ? corpus aggregation (pure; orchestration is in run_phase3_eval).
`::aggregate` (micro pool TP/FP/FN, TP-weighted mean_iou, macro F1), `::default_stages_for`
(`auto`; `::_FINAL_ONLY_SEGMENTERS={motion,gemini}`), `::resolve_video_path` (normalizes
collector `./data\..` Windows paths), `::format_aggregate`.
135:- `@backend/eval/training.py` ? learned-segmenter data layer. `::YOLO_FEATURE_NAMES` (5-d),
`::extract_yolo_features` (? reads `row['stats']` dict from DB; missing fields default 0.0),
`::label_candidates` (same IoU `match_intervals` as evaluator),
`::build_training_set`->`(X,y,intervals)`; `feature_fn` is the substrate plug-in seam.
138:[Omitted long matching line]
142:[Omitted long matching line]
155:[Omitted long matching line]
176:[Omitted long matching line]
177:[Omitted long matching line]
178:[Omitted long matching line]
208:$ **eval primitives**: `Interval=Tuple[float,float]` (start<end).
`Match{pred_index,gt_index,iou}`. `MatchResult{matches, unmatched_pred_indices(=FP),
unmatched_gt_indices(=FN)}`. `Score{iou_threshold,num_pred,num_gt,true_positives,false_positives
,false_negatives,precision,recall,f1,mean_iou,mean_start_error,mean_end_error}.as_dict()`.

```

```

214:[Omitted long matching line]
221:- `detectors/detector.py::TracknetV2Detector.run_tensor(imgs, affine_mats) -> (results,
hms_vis)`. `results[bid][eid]` = list of `{xy:np.array (ORIGINAL coords), score, scale}`.
(caller uses `detector.run_tensor(imgs, trans)` returning `(batch_results, _)` ? consistent.)
237:- **Config access:** `backend/main.py` and all four `run*.py` load via
`backend.config.load_config` (typed `AppConfig`). The legacy `.get()` dict-compat shim remains
for validators/segmenters. Remaining literal: a defensive `getattr(..., 'motion')` fallback in
`main.py` that only fires if `global_config`/.indexing` is `None` (the ASGI-import edge case).
238:- **Segmenter default:** single source of truth = model
`IndexingConfig.default_segmenter='yolo_hybrid'`; `config.json` may override at runtime (ships
`gemini`). `run*.py` read the model value (no literals); `main.py` keeps the defensive `motion`
getattr fallback noted above.
251:- **Twin segmenters:** `wasb_hybrid.py` / `tracknet_hybrid.py` are copy-paste; fixes to
P3/P4 must land in both. ? subtle drift: wasb's shot_count `int(None)`-via-TypeError fallback vs
tracknet's `is None` check.
264:| Change a config default / add a config field | `@backend/config/models.py` (single source
of truth) -> regenerate via `setup.py`/`save_default_config`; ? config.json may override
(default_segmenter) |
267:| Add a segmenter | subclass `@backend/pipeline/segmenters/base.py::BasePlaySegmenter`;
`segmenter_registry.register(...)` at module bottom; import in
`@backend/pipeline/segmenters/__init__.py`; if AI-based add to
`@backend/reporting/harvest.py::_AI_SEGMENTERS` |
287:| Train the learned segmenter | `@backend/eval/training.py::build_training_set` ->
`@backend/eval/classifier.py::LogisticRegression.fit/save` |
296:| `test_base.py` | `pipeline/segmenters/base.SegmenterRegistry` (register/get/list via
DummySegmenter) |
297:| `test_yolo_hybrid.py` | `pipeline/segmenters/yolo_hybrid.HybridYoloSegmenter`
(torchvision+ByteTrack; `_PERSON_CLASS_ID` filter; candidate link; `store_yolo_candidates=False`
skip) |
298:| `test_tracknet_hybrid.py` | `pipeline/segmenters/tracknet_hybrid.TrackNetHybridSegmenter`
(4-phase; healthcheck-abort; `_probe_fps_width` fallback) |
299:| `test_gemini.py` | `pipeline/segmenters/gemini.GeminiPlaySegmenter` (single vs chunked
path, per-chunk offset, `shot_count` <- `shuttle_exchange_count`) |
324:| `test_validators_base.py` | `pipeline/validators/base.ValidationRegistry`
(register/run_all/`run_all_with_context` -> `(results,context)`; exception captured not raised)
|
328:| (no dedicated test) | individual `pipeline/validators/*` impls,
`pipeline/stitcher.compiler`, `main.py`, `run_process_and_stitch.py`,
`pipeline/detectors/wasb_runner`, `pipeline/segmenters/{wasb_hybrid,motion}`,
`eval/audio/e1_probe` (diagnostic) ?? exercised only indirectly |

```

13. user (2026-06-10T06:10:39.612Z)

```

95 - ? `::AppConfig.get` ? legacy dict-compat shim: returns nested **sections as plain
dicts** (`model_dump(mode='json')`, NOT the model) so chained
`config.get("indexing",{ }).get(...)` keeps working; falls back to searching a leaf key in any
section (sections discovered dynamically from `model_fields`). ? "bit every validator on real
runs" per docstring ? returning the model breaks `.get`. `::AppConfig.__getitem__` raises
`KeyError` for unknown top-level keys else delegates to `.get`.
96 - `::IndexingConfig` ? segmenter/detector knobs (`motion_threshold=0.08`,
`min_segment_duration=3.0`, `dead_time_merge_gap=2.0`, `chunk_duration=90.0`,
`chunk_overlap=10.0`, `yolo_velocity_threshold=0.15`, `yolo_frame_skip=3`,
`ai_handoff_padding=2.0`, `ai_max_workers=5`, `log/store_yolo_candidates=True`, nested
`tracknet`). ? `default_segmenter='yolo_hybrid'` HERE but config.json overrides to `gemini`. ?
`::IndexingConfig.segmenter_must_be_registered` `@field_validator` is a **no-op pass-through** ?
a typo'd segmenter validates fine, only fails later at `segmenter_registry.get(...)`
97 - `::ValidationConfig` (`min_resolution_width=1280`, `min_resolution_height=720`, `min_f
ps=29.9`, `min_blur_threshold=20.0`, `max_scene_cuts_per_minute=0.5`, nested `relevance`);
`::ValidationRelevanceConfig` (court HSV gates, `court_pixels_min_ratio=0.05`).
`::TrackNetConfig` (WSL invocation contract: `wsl_distro='Ubuntu'`, `repo_dir`,
`conda_env='TrackNetV4'`, `weights_path='', `queue_length=5`, `velocity_threshold=0.02`).
`::OutputConfig` (`default_highlights_name`, `db_name='sports_indexer.db'`,
`output_dir='output'` ? CLI `--output-dir` overrides it on the typed model in `main()`).
`::StitchingConfig`
(`begin_padding=0.5`, `end_padding=0.5`, `use_cache=False`, `min_shot_count=1`). `::Sport` Literal

```

```

of 5 sports. `::Config = AppConfig` alias.
98     - `@backend/config/loader.py` ? load/save with defaults-merge.
99     - `::DEFAULT_CONFIG_PATH = Path("config.json")` (? CWD-relative, NOT repo-root).
`::get_builtin_defaults` = `AppConfig().model_dump(...)`
100    - `::load_config` ? precedence built-in-defaults -> file overrides. ? **shallow top-
level merge** (`{**defaults, **file_data}`): a section in config.json REPLACES the whole
defaults dict for that section (Pydantic then re-applies field defaults for missing leaves).
Missing/unreadable/parse-error file -> fully-defaulted AppConfig + `[CONFIG WARNING]` (non-
fatal).
101    - `::save_default_config` ? writes fresh `config.json`, **overwrites** if present
(used by setup.py/--setup). `::get_default_config` ? ? transitional plain-dict alias.
102    - `@config.json` ? on-disk config; mirrors `AppConfig`. ?
**indexing.default_segmenter='gemini'` ** diverges from model default `yolo_hybrid` ? file
wins at runtime.
103    - `@setup.py` ? `::init_default_config` ? delegates to
`backend.config.save_default_config` (single-source defaults; skips if config.json exists).
`::run_pip_install` GPU-aware (`nvidia-smi` -> cu124 else cpu). `::run_diagnostics` (Python?3.8,
ffmpeg/ffprobe on PATH, optional torch+CUDA).
104
105    ### 2.1 Segmenters (under pipeline/segmenters) ?
106    - `@backend/pipeline/segmenters/base.py`
107    - `::BasePlaySegmenter` ABC ? `__init__(db, config)`; abstract
`name`, `description`, `process_video`. & `process_video(video_id, video_path, player_pool=None,
max_frames=None) -> List[dict{id, start_time, end_time, duration, metadata}]`.
108    - `::SegmenterRegistry`, `::segmenter_registry` (singleton). ? **register a new
segmenter**: subclass `BasePlaySegmenter`, call `segmenter_registry.register(MyCls)` at module
bottom, AND import the module in `@backend/pipeline/segmenters/__init__.py` (registration is an
IMPORT side-effect). ? `register()` builds `cls(None, {})` just to read `name` ->
name/description/`__init__` MUST NOT touch `self.db`/`self.config`. ? `list_available()` re-
instantiates + swallows exceptions into "No description available." (a throwing `description` is
masked).
109    - `@backend/pipeline/segmenters/__init__.py` ? imports all 5
(`motion, gemini, yolo_hybrid, tracknet_hybrid, wasb_hybrid`) = the de-facto registration manifest;
re-exports them + `segmenter_registry` via `__all__`.
110    - `@backend/pipeline/segmenters/wasb_hybrid.py::WasbHybridSegmenter` (name
`wasb_hybrid`) ? WSL WASB substrate (MIT; ships pretrained badminton weights, the LIVE
default). Imports `WasbRunner`/`WasbConfig` from `pipeline/detectors/wasb_runner`. Config
`indexing.wasb.velocity_threshold(0.02)` + shared
`dead_time_merge_gap/min_action_window_duration/ai_handoff_padding/ai_max_workers`. ? whole-
video single pass; reuses YOLO-named keys `log_yolo_candidates/store_yolo_candidates`; needs
`{PROVIDER}_API_KEY` (missing -> []); `detection_params` has NO `queue_length` (unlike
tracknet). ? divergent shot_count: `int(shot_count)` inside try with no `is None` short-circuit
(copy-paste drift).
111    - `@backend/pipeline/segmenters/tracknet_hybrid.py::TrackNetHybridSegmenter` (name
`tracknet_hybrid`) ? copy-paste twin of wasb_hybrid; imports `TrackNetRunner`/`TrackNetConfig`
from `pipeline/detectors/tracknet_runner`; `indexing.tracknet.velocity_threshold(0.02)`;
`detection_params` adds `queue_length`. ? any P3/P4 fix must be applied to BOTH (and arguably
yolo_hybrid).
112    - `@backend/pipeline/segmenters/yolo_hybrid.py::HybridYoloSegmenter` (name
`yolo_hybrid`, no YOLO remains ? torchvision detector + supervision ByteTrack; ADR-005).
`::PERSON_CLASS_ID=1` ? (ultralytics used 0; wrong value -> zero detections).
`::lazy_load_model` (lazy `torchvision` import so module/tests load without it),
`::extract_action_windows_from_chunk`, `::make_tracker` (per-chunk ByteTrack ? track IDs
chunk-local), `::DETECTOR_FACTORIES`/`::DEFAULT_DETECTOR='fasterrcnn_resnet50_fpn'`. ?
velocity at `effective_fps = fps/frame_skip` -> `yolo_velocity_threshold` coupled to
`frame_skip`; pipelined-vs-sequential P2 (prefetch ffmpeg overlaps sequential GPU);
`yolo_tracker`/`bytetrack.yaml` is dead/back-compat. ? `gemini_*` config keys are deprecated
aliases still honored.
113    - `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter` (name `gemini`) ?
pure-AI, no P2. ? chunks only if `duration>chunk_duration`; `max_workers=5` hardcoded;
`max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates; `parse_time` accepts
colon timestamps (warns); does NOT persist candidate segments (only hybrids feed the fine-tuning
table).
114    - `@backend/pipeline/segmenters/motion.py::MotionPlaySegmenter` (name `motion`) ?
pixel absdiff baseline. ? **metadata + serve/smash/foul/winner events are RANDOM**

```

```
(random.sample/randint/uniform/choice); only segmenter using `db.add_event` and passing
server/receiver; uses `print()`; honors `max_frames`.
115
116     ### 2.2 Detectors & windowing (WSL-quarantined; ABC abstraction complete)
117     - `@backend/pipeline/detectors/base.py` ? DetectorRunner ABC (healthcheck() ->
Tuple[bool,str] never-raises; run_predict(...) -> Optional[str] CSV path). Contract deliberately
matches the pre-existing tuple returns and call sites in the hybrids. Rich module docstring
includes "how to add a new runner", Linux-native path notes (the de-risk goal), and guidance on
threading health into reports. Re-exported from `detectors/__init__.py`. This finishes the WSL
de-risk seam (low-regret 6 / review item 1).
118     - `@backend/pipeline/detectors/tracknet_runner.py`
119     - `::TrackNetConfig` (+`.from_indexing_cfg`; ? key drift `wsl_distro` JSON -> `distro`
field), `::TrackNetRunner` ? runs TrackNet `predict.py` in WSL.
```

14. user (2026-06-10T06:10:50.068Z)

docs\archives\COMMERCIAL_READINESS_REVIEW.md:54:The foundation (typed config, layered structure, Result contract, logging, packaging, Detector seam, observability reports as the quality surface, minimal value exports) is solid. We can now iterate on accuracy, data flywheel, and product value without fighting the code.

docs\archives\COMMERCIAL_READINESS_REVIEW.md:72: - Experiment with audio fusion (E1 probe results + hit_detector) as a cheap recall booster or candidate filter (ADR-007 context).

docs\DEFERRED_HARDENING_PLAN.md:9:remaining high-value hygiene items not already done (config/structure/Result/logging/

docs\DEFERRED_HARDENING_PLAN.md:98: returning `{status, progress, result, reports}`.

docs\PLATFORM_ARCHITECTURE.md:72: ? Orchestration / Jobs (submit?status?result) ?

docs\PLATFORM_ARCHITECTURE.md:143: pipeline, and writes artifacts + results back ? so **the orchestrator never holds the

docs\PLATFORM_ARCHITECTURE.md:222: status (queued|running|done|failed), progress, compute_backend, result_ref, error,

docs\PLATFORM_ARCHITECTURE.md:401:4. **Register** the result as the `owned\_vlm` provider (Gemini paid fallback stays) **only

docs\archives\decisions\DECISIONS.md:131: ffp probe the result and store the *actual* width/height/fps ? never assume.

docs\archives\decisions\DECISIONS.md:139: result never overwrites a good entry.

docs\CODE_MAP.md:127:- `@backend/eval/metrics.py` ? scoring primitive (THE spine). `::interval\_iou`, `::match\_intervals` (greedy, deterministic), `::score`->`::Score`(`.as\_dict() = wire shape), `::pr_curve`, `::Match`, `::MatchResult`. ? greedy not Hungarian; empty matches -> 0.0 (not NaN) for mean_iou/boundary errors.

docs\CODE_MAP.md:142:[Omitted long matching line]

docs\CODE_MAP.md:155:[Omitted long matching line]

docs\CODE_MAP.md:176:[Omitted long matching line]

docs\CODE_MAP.md:208:\$ **eval primitives**: `Interval=Tuple[float,float]` (start<end). `Match{pred\_index,gt\_index,iou}`. `MatchResult{matches, unmatched\_pred\_indices(=FP), unmatched\_gt\_indices(=FN)}`. `Score{iou\_threshold,num\_pred,num\_gt,true\_positives,false\_positives,false\_negatives,precision,recall,f1,mean\_iou,mean\_start\_error,mean\_end\_error}.as\_dict()`. docs\CODE_MAP.md:221:- `detectors/detector.py::TracknetV2Detector.run\_tensor(imgs, affine\_mats) -> (results, hms\_vis)`. `results[bid][eid]` = list of `{xy:np.array (ORIGINAL coords), score, scale}`. (caller uses `detector.run\_tensor(imgs, trans)` returning `(batch\_results, \_)` ? consistent.)

docs\CODE_MAP.md:324:| `test\_validators\_base.py` | `pipeline/validators/base.ValidationRegistry` (register/run_all/run_all_with_context -> `(results,context)`; exception captured not raised) |

docs\TRACKNET_WSL_SETUP.md:29: `~/models/TrackNetV4/docs/RESULT.md` are placeholders `( # )`. You will obtain

docs\VIDEO_RECORDING_GUIDELINES.md:57:1. **User-facing best-practices guide** ? short, friendly, illustrated: "Put the camera here, this high, record at ?X, keep it still." The goal is that a user can record once and get great results.

docs\BACKLOG.md:80:- ? **Item 3 (Error/Result contract) synergizes with the report.** Today the report INFERS failures (e.g. `silent\_provider\_noop` = 0 segments + no API key). A structured Result/error from providers/segmenters would let the report read the real reason as a first-class flag. **Revisit:** with item 3.

docs\archives\checkpoints\2026-06-eval-baseline\README.md:15:- Any existing P/R/F1 baselines or cross-validation results on the then-current golden corpus.

docs\archives\quality-iterations\README.md:13: - References to updated eval baselines, regression results, report enhancements, etc.

docs\archives\quality-iterations\README.md:18:- ****`2026-06-initial/`**** ? SEED. Opened the quality phase ahead of the awaited labeled-video batch (status: in progress; superseded for first results by the iteration below).

docs\archives\research\AUDIO_RALLY_DETECTION_PLAN.md:95:## 10. E1 RESULTS ? preliminary, on existing GoPro audio vs Gemini SILVER labels (2026-06-07)

docs\archives\checkpoints\2026-06-scaffolding-complete\README.md:8:This checkpoint captures the state of the project once the core scaffolding (typed config, layered structure, Result contract, logging/packaging, DetectorRunner abstraction, observability reports as the quality surface, and first value-layer exports) was mature enough to iterate on.

docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md:38:## Results ? first trustworthy human-GT numbers

docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md:33:3. Error / Result Contract standardization.

docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md:162:- Error / Result Contract standardization (small, high signal). ?

docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md:186:- Low-regret 1-6 (Config, structure, Result contract, logging, packaging, Detector Runner ABC finish ? #49)

docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md:195:- ****Item 8 (Input-Quality UX) ? mostly already delivered.**** The reports already surface validator results + footgun flags + a verdict (technical) ***and*** a clean customer summary. Reframe item 8 as ****extend the report**** (e.g. an audio-readiness signal in `backend/reporting/` + the obsreport spec), not a new surface.

docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md:197:- ****Item 3 (Error/Result contract) ? synergy.**** A structured Result from providers/segmenters lets the report read failure reasons directly instead of inferring them (e.g. `silent\_provider\_noop`).

docs\archives\research\OWNED_MODEL_TRAINING_STUDY.md:8:> synthesis + completeness critic; 35 sources). Empirical results from THIS session (LOVO + a learned

docs\archives\research\OWNED_MODEL_TRAINING_STUDY.md:42:TAS head, given ASFormer's small-data result used 2048-d I3D features?* This session answered it.

docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md:27:## Result 1 ? full-match Adarsh (96 rallies, IoU?0.5)

docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md:33:## Result 2 ? first cross-video LOVO (`calibrate\_local`, the number to trust)

docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md:49:## Result 3 ? learned per-frame prototype (the proof) ? `backend/eval/rally\_seq\_proto.py`

15. assistant (2026-06-10T06:11:01.269Z)

Important counter-evidence found in docs/BACKLOG.md:80 ? the report layer apparently infers `silent\_provider\_noop`. Let me check the reporting code and the low-regret plan scope.

16. user (2026-06-10T06:11:03.308Z)

```
backend\reporting\spec.yaml-19- - { id: stages,      title: "Processing stages",  kind: stages,
audience: technical }
backend\reporting\spec.yaml-20- - { id: findings,   title: "Findings & warnings", kind: flags,
audience: technical }
backend\reporting\spec.yaml-21- - { id: verdict,   title: "Verdict",              kind:
verdict,    audience: both }
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\spec.yaml-22-
backend\reporting\spec.yaml-23-rules:
backend\reporting\spec.yaml:24: # The classic silent failure: 0 segments because the AI
provider no-op'd on a
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\spec.yaml-25- # missing
API key ? NOT because the video has no rallies.
backend\reporting\spec.yaml:26: - code: silent_provider_noop
backend\reporting\spec.yaml-27- severity: error
backend\reporting\spec.yaml-28- when:
backend\reporting\spec.yaml-29- - { path: "stages.ai_handoff.details.api_key_present", op:
eq, value: false }
backend\reporting\spec.yaml-30- - { path: "stages.segmentation.metrics.segment_count", op:
eq, value: 0 }
backend\reporting\spec.yaml-31- message: >-
```

```

backend\reporting\spec.yaml:32:      0 segments AND no AI provider API key was set. The
segmenter (see Processing
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\spec.yaml-33-      stages
> segmentation > segmenter_actual) hands its candidate windows to the AI
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\spec.yaml-34-
provider, which silently did nothing without a key ? so this is a MISSING
backend\reporting\spec.yaml-35-      CREDENTIAL, not 'the video has no rallies'. Fix: put
GEMINI_API_KEY=<key> in a
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\spec.yaml-36-      .env
file in the project root and re-run (see README#Q7).
backend\reporting\spec.yaml-37-      faq_ref: "README#Q7"
--
backend\reporting\spec.yaml-46-      when:
backend\reporting\spec.yaml-47-      - { path: "stages.ai_handoff.details.ai_based", op: eq,
value: true }
backend\reporting\spec.yaml-48-      - { path: "stages.ai_handoff.details.api_key_present", op:
eq, value: true }
backend\reporting\spec.yaml-49-      - { path: "stages.segmentation.metrics.segment_count", op:
eq, value: 0 }
backend\reporting\spec.yaml-50-      message: >-
backend\reporting\spec.yaml:51:      An AI-based segmenter returned 0 segments even though the
API key is set.
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\spec.yaml-52-      Empty
can mean a provider error, exhausted quota, or (for WASB/TrackNet) a
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\spec.yaml-53-      failed
WSL healthcheck ? these all return [] silently. Re-run with -u for
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\spec.yaml-54-
unbuffered logs and check the provider/WSL output before concluding 'no rallies'.
backend\reporting\spec.yaml-55-      faq_ref: "README#Q7"
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\spec.yaml-56-
--
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\harvest.py-135-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\harvest.py-136-
backend\reporting\harvest.py-137-def _ai_stage(rec: "obsreport.RunRecorder", config: Dict[str,
Any], segmenter_actual: Optional[str],
backend\reporting\harvest.py-138-      play_segments: List[Dict[str, Any]], ran: bool)
-> None:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\harvest.py-139-      """Only
meaningful for AI-based segmenters. Records whether the provider key was
backend\reporting\harvest.py:140:      present ? the signal the silent_provider_noop rule keys
on."""
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\harvest.py-141-
ai_based = segmenter_actual in _AI_SEGMENTERS
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\harvest.py-142-
provider = (config.get("ai_provider") or "gemini")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\harvest.py-143-      key_env
= f"{provider.upper()}_API_KEY"
backend\reporting\harvest.py-144-      with rec.stage("ai_handoff") as st:
backend\reporting\harvest.py-145-      if not ran or not ai_based:

```

17. user (2026-06-10T06:11:04.556Z)

```

25      **Rule:** Every item must be traceable to a concrete pain/risk from the Commercial
Readiness Review or existing BACKLOG/ADRs.
26
27      ---
28
29      ## 2. Overall Sequence (Approved Order)
30
31      1. **Config Model (Pydantic) + Central Loader** (as a proper package) ? foundational.
32      2. **Directory Structure Refactor** (high-pri, atomic follow-up PR) ? move to the
approved layered layout.
33      3. Error / Result Contract standardization.
34      4. Logging standardization + removal of stray `print()` in hot paths. ? (done in
feat/logging-standardization; central setup in main; replaced in motion.py, wasb_infer.py,

```

```

stitcher/compiler.py, config/loader.py, main internal diagnostics; user-facing output in
run_*.py and main CLI kept as print where appropriate for direct console UX. Docs updated in
CODE_MAP and this plan.)
35     5. Packaging modernization (`pyproject.toml` + clean entry points). ? (done in
feat/packaging-modernization; pyproject.toml with PEP 621 + [project.scripts]; pip install -e .
supported; setup.py retained for diagnostics; docs + tests updated.)
36     6. Detector Runner Abstraction (start + finish de-risking WSL). ? (start sketched;
finished in feat/finish-detector-runner-abstraction + PR #49: base.py ABC now matches the real
(ok, msg) contract; runners inherit; hybrids type against DetectorRunner; contract tests for
both; rich docs for adding Linux-native runners. 224 passed @64%.)
37     7. Unify / extract shared logic from the two hybrid segmenters.
38     8. Input Quality UX ? reframed as extend existing reports (audio readiness + capture
guidance from VIDEO_RECORDING_GUIDELINES). ? (feat/extend-reports-input-quality + PR #50,
merged)
39     Value Layer (exports EDL/JSON/CSV timelines + basic stats as minimal first slice). ?
(feat/value-layer-minimal-exports + PR #51, merged)
40     9. (Stretch) First slice of async job model.
41
42     **Important sequencing decision (approved):** Complete the Config work **first** (this
introduces `backend/config/` as a package immediately). Then perform the broader directory
refactor as a **separate, atomic, high-priority PR**.
43
44     Rationale:
45     - Keeps the Config PR small and focused.
46     - Allows the structural change to land atomically with minimal merge conflicts.
47     - Gives Claude a clean window with no concurrent file moves.
48     - Still delivers the "level of indirection" benefit early via the `config/` package.
49
50     ---
51
52     ## 3. Approved Directory Structure Direction
53
54     Current flat layout under `backend/` (many sibling dirs + root .py files) feels
unprofessional for long-term maintenance. We will trend toward clear layers without creating 15+
tiny single-file directories.
55
56     **Target layered structure (directional, not all at once):**
57
58     ```
59     backend/
60     ??? __init__.py
61     ??? main.py                # Thin CLI + FastAPI bootstrap only
62     ?
63     ??? config/                # ? Start here with Deliverable 1
64     ?   ??? __init__.py
65     ?   ??? models.py          # Pydantic AppConfig + nested models
66     ?   ??? loader.py          # load_config, save_default_config, etc.
67     ?
68     ??? api/                   # Presentation layer
69     ?   ??? __init__.py
70     ?   ??? app.py             # FastAPI instance, middleware, routes
71     ?   ??? static/            # Frontend assets (HTML/JS/CSS)
72     ?
73     ??? pipeline/              # Core processing engine
74     ?   ??? validators/
75     ?   ??? segmenters/        # (renamed/moved from current indexer/)
76     ?   ??? stitcher/
77     ?   ??? audio/
78     ?   ??? detectors/
79     ?
80     ??? infrastructure/        # External adapters
81     ?   ??? database.py        # (moved from root)
82     ?
83     ??? providers/             # AI providers (group under infrastructure later if
desired)

```


18. user (2026-06-10T06:11:12.959Z)

```

140    ### 4.5 Success Criteria
141    - Typed config is the only way new code reads settings.
142    - Existing `config.json` files (and `setup.py` output) continue to work unchanged.
143    - `backend/config/` package exists and sets the pattern for the larger structure.
144    - One focused, reviewable PR.
145
146    ---
147
148    ## 5. Next High-Priority Item: Directory Structure Refactor (Separate Atomic PR)
149
150    After the Config PR lands and is verified:
151    - Create a dedicated branch/PR whose only job is the layout change.
152    - Move files into `api/`, `pipeline/`, `infrastructure/` etc.
153    - Update all imports, entry points, docs (`CODE_MAP.md`, README, tests), and any hard-
154    coded paths.
155    - Goal: one clean atomic change with no one else (Claude) needing to resolve conflicts
156    at the same time.
157
158    This item is now tracked at the top of the sequence and in BACKLOG.
159
160    ---
161
162    ## 6. Later Low-Regret Items (after the above two)
163
164    - Error / Result Contract standardization (small, high signal). ?
165    - Logging standardization. ?
166    - Packaging modernization. ?
167    - Detector Runner Abstraction (WSL de-risk). ? (feat/finish-detector-runner-abstraction)
168    - Unify hybrid segmenter logic.
169    - Input Quality UX surfacing.
170    - Async job model slice.
171
172    See BACKLOG for status tracking.
173
174    ---
175
176    **Checkpoint note (2026-06-07 + post-merge update):**
177    The directory structure proposal and the "Config first, then atomic refactor" sequencing
178    decision were reviewed and approved.
179
180    Low-regret items 3?6 + the three scaffolding maturation items (Detector finish #49,
181    Reports-based Input Quality #50, minimal Value Layer #51) all completed with per-item
182    branches/PRs, full miniconda test + coverage runs (maintained ~64%), and docs updates.
183
184    User merged #50 and #51. Scaffolding phase complete; shift focus to quality improvement
185    with new labeled videos (expected mid-next week). See refined priorities in
186    COMMERCIAL_READINESS_REVIEW.md.
187
188    ---
189
190    ## 7. Status & observability-feature intersections (2026-06-07, refreshed)
191
192    **Done (merged):**
193    - Low-regret 1-6 (Config, structure, Result contract, logging, packaging, Detector
194    Runner ABC finish ? #49)
195    - Scaffolding maturation batch:
196      - 2. Input Quality UX via reports extension (audio readiness signal + capture guidance
197      surfaced in reports from VIDEO_RECORDING_GUIDELINES) ? #50
198      - 3. Value Layer (minimal first slice: EDL/JSON/CSV timeline exports + basic stats
199      over play segments) ? #51
200

```

191 All three recommended scaffolding items completed as separate PRs with full
tests/coverage/docs discipline. Scaffolding now matured for iteration.
192
193 In parallel, **per-video observability reports** shipped across both tools (indexer PR
#41/#42/#43, collector PR #11/#12) on a new shared, permissively-licensed lib **obsreport** (a
small "report config language"). Coordination so the remaining items don't collide or duplicate:
194
195 - **Item 8 (Input-Quality UX)** ? mostly already delivered. The reports already surface
validator results + footgun flags + a verdict (technical) *and* a clean customer summary.
Reframe item 8 as **extend the report** (e.g. an audio-readiness signal in `backend/reporting/``
+ the obsreport spec), not a new surface.
196 - **Item 7 (unify hybrid segmenters)** ? do this before wiring real AI-provider timing. The
report records AI timing as `not_captured`; a unified segmenter gives one place to thread
the timing in. (Claude will add that hook with/after item 7.)
197 - **Item 3 (Error/Result contract)** ? synergy. A structured Result from
providers/segmenters lets the report read failure reasons directly instead of inferring them
(e.g. `silent_provider_noop`).
198 - **Items 4 (logging), 5 (packaging), 6 (detector runner abstraction)** ? safe to do
independently (near-zero overlap with the reports). For #5, model `pyproject.toml` on
`obsreport`'s and pin the `obsreport` dep tag.
199
200 Also added for the handoff: **run_all_tests.py** at the repo root ? one-command full
suite (clear PASS/FAIL + pytest exit code) before opening PRs.
201
202 Found while validating the restructure end-to-end: the observability report **surfaced a**
real config runtime bug (the typed `AppConfig` broke validators' nested `.get`), fixed in PR
#42 ? a good signal that the reporting layer earns its keep.

19. user (2026-06-10T06:11:22.629Z)

50 The exact grouping **metric is the owner's design choice** (consolidate small files
vs. add per-area dirs ? the
51 "lesser fragmentation" goal slightly tensions with adding dirs), so this is left as a
dedicated refactor PR,
52 not guessed here. **Revisit:** when the owner specifies the grouping scheme. (The
`backend/eval/audio/`` half of
53 the request ? comment 1 ? was done in PR #35.)
54
55 **## Commercial Readiness & Low-Regret Moves ? Approved Decisions Checkpoint (2026-06-07)**
56
57 ***(source: Commercial Readiness Review + structure/sequencing discussion with Grok as**
second-in-command)*
58
59 - ? **Docs-only checkpoint PR** ? This update + the refreshed
`docs/LOW_REGRET_MOVES_PLAN.md` formally record the approved path before any code changes. The
initial review docs (PR #37) introduced the findings; this entry locks the execution decisions.
60 - ? **Approved directory structure** ? Adopt a layered approach under `backend/`` for
professionalism and future growth (no flat dumping of new modules, no explosion of 15+ single-
file dirs):
61 - `backend/config/`` (package ? start with this)
62 - `backend/api/`` (FastAPI app, routes, static UI)
63 - `backend/pipeline/`` (validators + segmenters + stitcher + audio + detectors)
64 - `backend/infrastructure/`` (database + external adapters)
65 - Keep `backend/eval/`` prominent as a first-class concern.
66 - Logical grouping for providers, sports, annotations, utils, and a small `core/`` only
when needed.
67 - ? **Sequencing (Config first, then atomic refactor)** ? Implement Deliverable 1
(Config) **first**, introducing the `backend/config/`` package immediately. Perform the full
directory structure refactor as a **separate high-pri atomic PR** afterwards. This minimizes
conflicts, keeps PRs focused, and gives Claude a clean window with no concurrent moves.
68 - ? **Deliverable 1 (Config)** ? `backend/config/`` package (`models.py` + `loader.py`)
instead of a single `config.py`. Typed `AppConfig`, central loader with built-in defaults + file
override, `extra="allow"` for migration. Update `setup.py` + `main.py`. Add tests. Land this
before the layout move.
69 - ? **Subsequent items** ? See the refreshed `LOW_REGRET_MOVES_PLAN.md` for the full

ordered list (Error contracts, logging, packaging, detector abstraction, etc.). The structure refactor is now explicitly tracked as its own high-pri step right after Config.

```

70
71     **Status legend reminder:** ? = approved / in progress tracking, ? = do when triggered.
72
73     ## Observability reports ? shipped; intersections with the low-regret items (2026-06-07)
74     *(source: per-video observability feature ? indexer PR #41/#42/#43, collector PR
75     #11/#12; shared lib `obsreport`)*
76     - ? **Low-regret items 1 (Config pkg, PR #39) + 2 (directory restructure, PR #40) are
77     DONE.** Remaining = items 3?9 in `LOW_REGRET_MOVES_PLAN.md`.
78     - ? **Per-video observability reports shipped in BOTH tools** (a TECHNICAL report + a
79     CUSTOMER-friendly summary), via the **soft-dependency** lib `obsreport`
80     (github.com/avidullu/sports-obsreport, MIT, v0.1.3). Each run writes `<video>.report.json /
81     .report.md / .summary.md` next to the video.
82     - ? **Item 8 (Input-Quality UX) is LARGELY SUBSUMED** by these reports (validation stage
83     + footgun flags + verdict already surface quality; a clean customer summary already exists).
84     **Do NOT build a parallel mechanism** ? instead EXTEND the report: add an audio-readiness
85     stage/flag to `backend/reporting/` (harvest) + the obsreport spec. **Revisit:** when item 8 is
86     picked up.
87     - ? **Item 7 (unify hybrid segmenters) should land before the report's real AI-provider
88     timing.** The report records AI timing as `not_captured`; once `wasb_hybrid`/`tracknet_hybrid`
89     are unified there is ONE place to thread a recorder callback through `provider.analyze_video` ?
90     the report. **Revisit:** with item 7 (Claude to add the timing hook there).
91     - ? **Item 3 (Error/Result contract) synergizes with the report.** Today the report
92     INFERS failures (e.g. `silent_provider_noop` = 0 segments + no API key). A structured
93     Result/error from providers/segmenters would let the report read the real reason as a first-
94     class flag. **Revisit:** with item 3.
95     - ? **`obsreport` is a SOFT/commented dep** in `requirements.txt` (works locally via
96     editable install; pipeline no-ops if absent). **Pin the `git+ssh@v0.1.3` tag** in both repos
97     once SSH/CI access to the private lib repo exists.
98     - ? **`run_all_tests.py`** added at repo root ? run the full suite (clear PASS/FAIL,
99     pytest exit code) before sending PRs.
100    - ? **Config-migration follow-ups (DONE ? config & identity consistency pass)** ? the
101    two typed-config bugs from
102    #39/#40 are fixed: (1) **`/api/compile` now honors `--output-dir`** and (2)
103    **`output_dir` is a real field on
104    `OutputConfig`** (default `"output"`); the CLI overrides it on the typed model in
105    `main()`, retiring the
106    write-only `_runtime_overrides` hack. Same pass also unified `video_id` MD5 derivation
107    into
108    `backend/utils/hashing.py::compute_video_id` and migrated all four `run*.py` scripts
109    off raw `json.load` onto
110    `backend.config.load_config`. See `CODE_MAP.md` §2.0/§6 (footguns retired).
111
112

```

20. user (2026-06-10T06:11:32.026Z)

```

1     """Lightweight Result type for standardizing success vs. failure across the pipeline.
2
3     This is the start of the Error/Result Contract standardization (low-regret item #3).
4
5     The goal is to make it impossible to confuse "no data / no rallies" with "something
6     actually failed"
7     (e.g. missing API key, WSL healthcheck failure, provider error, etc.).
8
9     We use simple dataclasses so callers can use isinstance checks or structural pattern
10    matching.
11
12    This is intentionally small and incremental ? we start by applying it to the segmenter
13    and provider layers (the biggest sources of silent [] returns today).
14
15    The types are generic where it makes sense. Callers can still access .value on Success
16    or .message on Failure.
17
18    This module has no dependencies on the rest of the app.

```

```

16     """
17
18     from __future__ import annotations
19
20     from dataclasses import dataclass, field
21     from typing import Generic, TypeVar, List, Optional, Any, Union
22
23     T = TypeVar("T")
24
25
26     @dataclass
27     class Success(Generic[T]):
28         """Represents a successful outcome with an optional list of non-fatal warnings."""
29
30         value: T
31         warnings: List[str] = field(default_factory=list)
32
33         def is_success(self) -> bool:
34             return True
35
36         def unwrap(self) -> T:
37             """Return the value. Only safe to call if you already know it's a Success."""
38             return self.value
39
40
41     @dataclass
42     class Failure:
43         """Represents a failure with a human-readable message and optional details."""
44
45         message: str
46         details: Optional[dict[str, Any]] = None
47         is_recoverable: bool = False # e.g. missing API key (recoverable) vs internal crash
48
49         def is_success(self) -> bool:
50             return False
51
52         def unwrap(self) -> Any:
53             """Raise to make misuse obvious in code that assumes success."""
54             raise RuntimeError(f"Attempted to unwrap a Failure: {self.message}")
55
56
57     # Convenient alias for use in type hints: Result[list[dict], Failure]
58     # Result[ValueType] means Success[ValueType] | Failure
59     # (the Failure "error" type is fixed for this contract)
60     Result = Union[Success[T], Failure]
61

```

21. user (2026-06-10T06:11:34.193Z)

docs\BACKLOG.md:32:- ? **GS-3 ? `HumanVendorProvider`** ? concrete adapter for the chosen annotation vendor (submit/poll/fetch + DPA/secure-link controls). **Vendor selected for the initial pilot: Roora** (Bengaluru; iMerit / Taskmonk / TELUS remain the fallback shortlist). **Revisit:** once the Roora pilot confirms quality, then build the adapter.

docs\BACKLOG.md:35:- ? **Vendor pilot (Roora)** ? run the paid pilot with **Roora** as a **single-vendor pilot first** (not a multi-vendor bake-off): assess quality-per-rupee against the \$4 bar; escalate to the iMerit/Taskmonk fallback only if it misses. **Revisit:** the pilot doubles as calibration once the gold key exists.

docs\COMMERCIALIZATION.md:41:| C8 | **Golden-set vendoring spend** | **P1** | ? | ADR-006: **Roora (Bengaluru)** selected for an initial paid pilot; licensed/owned video only (no user uploads to vendors). | Confirm quality-per-dollar from the pilot; then GS-3 `HumanVendorProvider`. See `GOLDEN\_SET\_VENDORING.md`. |

docs\COMMERCIALIZATION.md:149: local hosting/partners; Denmark/EU small but high-WTP, easy to pilot.

docs\COMMERCIALIZATION.md:222:- `GOLDEN\_SET\_VENDORING.md` ? vendor process + Roora pilot (C8).

docs\GOLDEN_SET_IMPLEMENTATION.md:8:[Omitted long matching line]

docs\GOLDEN_SET_VENDORING.md:10:[Omitted long matching line]

docs\GOLDEN_SET_VENDORING.md:53:### Step 2 ? Run the bake-off (parallel pilot)

docs\GOLDEN_SET_VENDORING.md:54:Send the **same** pilot clips to **2?3** shortlisted vendors (see §6, favoring Bangalore/India), withholding the key. Grade every submission with **our own harness**:

docs\GOLDEN_SET_VENDORING.md:141:| **iMerit** | **Bengaluru** office (A101 & A118, 43 Residency Rd, 560025; tech hub opened Oct 2022); also Kolkata HQ + US | Yes ? image/video/text/audio; explicit Sports & Biomechanics domain on their site | Largest & most enterprise-ready (5,500+ annotators); has a real **Bangalore** footprint for in-person coordination; ask for a sports/temporal-segmentation reference + a paid pilot. Higher-touch ? likely higher cost. | **High** |

docs\GOLDEN_SET_VENDORING.md:142:| **TELUS International AI** (ex-**Playment**) | Playment was **Bangalore-founded** (2015); acquired by TELUS 2021, team retained | Yes ? built on Playment's 2D/3D image+video+LiDAR CV pipeline | Strong CV/video pedigree, Bangalore roots; but now a large global BPO ? confirm they'll take a **small startup pilot** and who the India delivery contact is. | **High** (origin) / **Med** (current India ops shape) |

docs\GOLDEN_SET_VENDORING.md:143:| **Taskmonk** | **Bengaluru** | Yes ? "Video" listed as a labeling modality; platform **managed-service** workforce | Mid-size, India-local, does both tooling and service ? potentially the best **cost/agility** fit for a startup pilot. Verify temporal/event labeling specifically (their heritage is e-commerce catalog tagging). | **Med-High** |

docs\GOLDEN_SET_VENDORING.md:149:| **Infolks** | India (Kerala-based per third parties) | Targets startups/SMBs; image/video/text/audio; positions on cost-efficiency ? good cheap-pilot candidate | **Med** |

docs\GOLDEN_SET_VENDORING.md:152:| **Roora** (Roora ML Pvt Ltd) | **Bengaluru** ? HSR Layout (verified, `rooragr.com`) | Image/video/text annotation; sports industry listed. **Selected for the initial paid pilot (2026-06-02)** ? existence/capability/location verified; **quality TBD via the pilot**. | **Verified** (capability) · pilot pending |

docs\GOLDEN_SET_VENDORING.md:158:- **Sports/temporal track record** ? beyond iMerit's stated "Sports & Biomechanics" domain, vendors' specific **rally-boundary / temporal-event** experience is unconfirmed marketing copy; probe with the pilot (§3).

docs\GOLDEN_SET_VENDORING.md:162:For the **offline conversations**, start with **iMerit** (Bangalore-local, enterprise QA, sports domain) and **Taskmonk** (Bangalore-local, likely more startup-friendly cost/agility) as the two primary contacts, with **TELUS/Playment** as a third if you want a heavyweight CV pedigree. Run the §3 bake-off across whichever 2?3 agree to a paid pilot, and decide on measured quality-per-rupee. **Verify every "Confidence: Med/Low" claim on the call before committing.**

docs\GOLDEN_SET_VENDORING.md:164:> **Update (2026-06-02):** the founder opted to start with **Roora** (Bangalore-local, startup-friendly) as a **single-vendor pilot** rather than the iMerit/Taskmonk multi-vendor bake-off above. iMerit / Taskmonk / TELUS remain the fallback shortlist if the Roora pilot doesn't clear the §3/§4 quality bar.

docs\GOLDEN_SET_VENDORING.md:182:[Omitted long matching line]

docs\PLATFORM_ARCHITECTURE.md:133: backend wraps today's behavior so nothing breaks; adapters land incrementally.

docs\PLATFORM_ARCHITECTURE.md:149: demand (an academy with an AWS account but no standing GPU box), add **SkyPilot**

docs\PLATFORM_ARCHITECTURE.md:157: treat `byo\_worker`/SkyPilot as a **power-user / enterprise / privacy-sensitive tier**,

docs\PLATFORM_ARCHITECTURE.md:357: Unsloth) on a **rented GPU** (RunPod / Lambda / Modal / SkyPilot-to-your-cloud): native

docs\PLATFORM_ARCHITECTURE.md:555:[Omitted long matching line]

docs\PMF_MARKET_RESEARCH.md:178:price-sensitive; Denmark/EU small but high-WTP and easy to pilot.

docs\PMF_MARKET_RESEARCH.md:199: price-sensitive, hard to bill). **Validate:** local pricing/payments pilot; per-academy

docs\ROADMAP.md:114:> initial **3-sport pilot** (badminton, pickleball, table tennis). Input-capture quality requirements

docs\archives\decisions\DECISIONS.md:285:### Update (2026-06-02): shortlist researched; Roora selected for the initial pilot

docs\archives\decisions\DECISIONS.md:288:- The founder selected **Roora ML Pvt Ltd** (HSR Layout, Bengaluru) for the **initial paid pilot**, diverging from the doc's iMerit/Taskmonk default toward a Bangalore-local, startup-friendly shop. Roora's existence, image/video/temporal capability, and location are verified (rooragr.com); **pilot quality is still to be measured** ? run as a **single-vendor pilot first** (the pilot **is** the calibration), not a multi-vendor bake-off.

docs\archives\decisions\DECISIONS.md:290:- ****Pilot scope is 3 sports**** (badminton, pickleball, table tennis); the collector's `import-local` ingestion for all three is implemented (collector ADR-001).

22. user (2026-06-10T06:11:51.002Z)

```
backend\api\static\app.js:115:          // Successfully processed
backend\api\static\app.js:121:          alert(`Success! Successfully processed and indexed
${indexedRallies.length} badminton rallies.`);
backend\main.py:31:from backend.results import Result, Failure # standardized error/result
contract
backend\main.py:139:          if isinstance(result, Failure):
backend\main.py:155:          "message": f"Successfully indexed {len(segments)} play segments
using '{seg_name}' segmenter.",
backend\main.py:399:          if isinstance(result, Failure):
backend\results.py:12:The types are generic where it makes sense. Callers can still access
.value on Success
backend\results.py:13:or .message on Failure.
backend\results.py:27:class Success(Generic[T]):
backend\results.py:37:          """Return the value. Only safe to call if you already know it's a
Success."""
backend\results.py:42:class Failure:
backend\results.py:54:          raise RuntimeError(f"Attempted to unwrap a Failure:
{self.message}")
backend\results.py:57:# Convenient alias for use in type hints: Result[list[dict], Failure]
backend\results.py:58:# Result[ValueType] means Success[ValueType] | Failure
backend\results.py:59:# (the Failure "error" type is fixed for this contract)
backend\results.py:60:Result = Union[Success[T], Failure]
backend\pipeline\stitcher\compiler.py:181:          logger.info(f"Successfully trimmed
{len(temp_clips)}/{len(segments)} clips. Preparing to stitch...")
backend\pipeline\segmenters\base.py:4:from backend.results import Result # new standardized
contract (Result[Value] == Success[Value] | Failure)
backend\pipeline\segmenters\base.py:29:          Use isinstance(result, Failure) (or
result.is_success()) to distinguish
backend\pipeline\segmenters\motion.py:9:from backend.results import Result, Success, Failure #
standardized contract
backend\pipeline\segmenters\motion.py:33:          return Failure(message=f"Could not open
video: {video_path}", is_recoverable=False)
backend\pipeline\segmenters\motion.py:226:          return Success(value=segments_data)
```

23. user (2026-06-10T06:11:52.051Z)

```
38      customer_message: >-
39      We couldn't generate highlights for this video due to a configuration issue
40      on our side. Please contact support.
41
42      # API key WAS present but an AI segmenter still produced nothing ? likely a
43      # provider error / quota / WSL healthcheck issue rather than a truly empty video.
44      - code: ai_empty_vs_no_rallies
45      severity: warn
46      when:
47      - { path: "stages.ai_handoff.details.ai_based", op: eq, value: true }
48      - { path: "stages.ai_handoff.details.api_key_present", op: eq, value: true }
49      - { path: "stages.segmentation.metrics.segment_count", op: eq, value: 0 }
50      message: >-
51      An AI-based segmenter returned 0 segments even though the API key is set.
52      Empty can mean a provider error, exhausted quota, or (for WASB/TrackNet) a
53      failed WSL healthcheck ? these all return [] silently. Re-run with -u for
54      unbuffered logs and check the provider/WSL output before concluding 'no rallies'.
55      faq_ref: "README#Q7"
56
57      # fps was assumed rather than probed -> velocity thresholds + timing may be wrong.
```

24. user (2026-06-10T06:12:32.432Z)

Structured output provided successfully

25. assistant (2026-06-10T06:12:40.422Z)

Verification complete.